

Projektreport „Textadventure“

Komponenten des Spiels:

- Scenes (Ordner mit Textdateien; Enthalten Szenen-Beschreibungen)
- `scenes.py` (Szenen-Objekte)
- `inventory.py` (Inventar und Items)
- `game.py` (Game-Loop)
- `alice_riddle.py` (Modul für Alice-Rätsel)
- `animal_riddle.py` (Modul für Tier-Rätsel)
- `animals.csv` (Textdatei zur Generierung des Tier-Rätsels)
- `textadventure.py` (Main-Datei, wird ausgeführt zum Start des Spiels)

Die Szenen des Spiels, sind als Klassen innerhalb eines eigenen Moduls angelegt. Jede Szene muss neben universellem Input auch speziellen verarbeiten und jede Szene hat unterschiedliche Folgeszenen. Für die Namensgebung, Beschreibung und das Verarbeiten des universellen Input wurde einer Super-Klasse angelegt, da dies in allen Szenen identisch ist. Der Name wird bei der Inszenanzierung als String mitgegeben, die Beschreibung wird aus Textdateien ausgelesen, deren Name dem der Instanz entspricht: Die Szene `scenes.TrainStation(„hbf“)` erhält ihre Beschreibung beispielsweise aus der Datei `hbf.txt`. Diese Textdateien sind im Ordner Scenes abgelegt. Dies ermöglicht eine unkomplizierte Änderung der Szenen-Beschreibung ohne die Funktionalität des Codes zu berühren.

Ausgehend von der Super-Klasse der Szenen, wird für jede der möglichen Szenen des Spiels je eine weitere Klasse erstellt, welche die Attribute und Funktionen der Hauptklasse erbt. Zusätzlich dazu enthalten die Sub-Klassen noch die Funktion die den szenenspezifischen Userinput bearbeitet und ggf. die aktuelle Szene wechselt .

Dadurch, dass die Szenen als Klassen angelegt sind, kann man für sie mit eigene, komplexe Funktionen definieren. Diese Funktionen lassen sich in der Game-Loop übersichtlich als Methode für eine Szenen-Instanz aufrufen, der Code bleibt übersichtlicher. Ein weiterer Vorteil ist, dass man bei Be-

darf innerhalb des Szenen-Moduls leicht eine neue Instanz erzeugen und in das Game einbinden kann.

Auch das Inventar setzt sich aus Klassen zusammen. Besonders das Geld des Users als Objekt einer Klasse mit definiertem numerischen Wert anzulegen, ermöglicht die Erstellung einer `.pay()` Methode auf der Klasse, was das „bezahlen“, das an anderen Stellen im Spiel stattfindet, vereinfacht.

Zuletzt ist auch das Textadventure selbst eine Klasse, auf welche die Methode `.play()` definiert wurde. Die Methode beinhaltet die Game-Loop. Ein Spieldurchlauf kann durch die Insanziierung eines Objektes der Klasse `Game()`, auf welches die `.play()` Methode angewendet wird gestartet werden. Innerhalb der Game-Loop findet erst eine Überprüfung statt, ob die Loop noch aktiv ist. Dann erfolgt die Überprüfung der aktuellen Szene: Erfordert diese ein Rätsel, dann wird das jeweilige Rätsel Modul aufgerufen und im Fall des Alice Rätsels ein Timer (durch die Verwendung des `time` Moduls) gestartet/gestoppt und die verbrauchte Zeit evaluiert. Wenn die aktuelle Szene ein Endpunkt ist, wird der User gefragt ob er erneut spielen will und entsprechend die Loop beendet oder eine neue Loop mit neuer Instanz des Games gestartet. Sind die Überprüfungen auf Rätsel- und Endszene abgeschlossen und keiner der Fälle liegt vor, wird der Userinput empfangen und gemäß der Klasse der aktuellen Szene verarbeitet.

Die Erstellung des Spiels als Klasse bietet den Vorteil, dass bei jedem neuen Spieldurchlauf das Spiel stets als neue Instanz mit den selben Start-Attributen aufgerufen wird.

Die Rätsel sind jeweils als eigenes Modul angelegt. Sie können in der Game-Loop aufgerufen werden, was den Code der Loop übersichtlicher macht. Die Rätsel Module selbst besitzen eine zweigeteilte Struktur. Es gibt eine Funktion, die einen Grundlagen-Text ausliest und daraus die Parameter für das Rätsel generiert und ausgibt und eine Funktion für das Lösen des Rätsels durch den User, welche zu beginn die erste Funktion aufruft um Parameter zu erhalten. Dies ist hilfreich, da die Rate-Loop es erforderlich machen kann, wiederholt neue Parameter zu generieren. Innerhalb der Rätsel-Generierung werden das `random` Modul sowie das `nltk` Modul genutzt. Im Fall des Alice-Rätsels wird mittels `nltk` Moduls der Alice Text aus dem Gutenberg Korpus ausgelesen und tokenisiert um danach mittels `random` Moduls einen Satz als Rätsel-Satz zu bestimmen der mit der `nltk` eingenen Methode mit POS-Tag versehen wird. Hier wird wiederum erneut durch das `random` Modul ein Zufallswert generiert der das zu erratende Wort auswählt. Für das Tier-Rätsel wird eine zufällige Zeile aus einer Textdatei ausgelesen und mittels `nltk` eine Definition bei Wordnet abfragt.

Die Obergrenzen bei der Generierung der Zufallswerte sind dabei variabel und richten sich nach Anzahl der Sätze/Satzlänge (Alice) und Zeilen der Textdatei (Tiere). Die beiden Rätsel-Module sind demnach so angelegt, dass die Rätsel-Grundlagen (Alice-Text und Tier-Liste) durch strukturell gleiche Dateien austauschbar sind ohne dass der Code sonst verändert werden muss.

Die auszuführende Main-Datei des Spiels (`textadventure.py`), erzeugt eine Instanz der Game-Klasse und wendet die `.play()` Methode an, was dem Starten des Spiels entspricht.